LAPORAN PROGRESS TUGAS AKHIR

Perancangan Sistem Electronic Nose Berbasis ESP32-S3 dengan Infrastruktur CSV Profiler, OTA Update, dan Simulasi Telemetri ThingsBoard

Disusun oleh:

Moreno Rizkihakiki	NRP. 2042241088
Muhammad Alif Al Ayyubi	NRP. 2042241070
Mochammad Ilham Fandy R.	NRP. 2042241107
Gatam Arbi Muhammad	NRP. 2042241100

Departemen Teknik Instrumentasi

Fakultas Vokasi

Institut Teknologi Sepuluh Nopember

Surabaya

30 September 2026

Daftar Isi

1 Pendahuluan **1**
- 1.1 Latar Belakang . . . 1
- 1.2 Tujuan . . . 1
- 1.3 Ruang Lingkup . . . 1

2 Arsitektur Sistem **3**
- 2.1 Gambaran Umum . . . 3
- 2.2 Komponen Backend . . . 3
  - 2.2.1 CSV Profiler Server (Python / Flask) . . . 3
  - 2.2.2 OTA Firmware Server (Rust / Axum) . . . 3
  - 2.2.3 Firmware Simulasi (Rust / ESP32-S3) . . . 4
- 2.3 Cara Menjalankan Sistem . . . 4

3 CSV Data Profiler **5**
- 3.1 Gambaran Fitur . . . 5
- 3.2 Tampilan Dashboard . . . 5
- 3.3 Hasil Profiling Dataset Sensor . . . 6
  - 3.3.1 Ikhtisar Dataset . . . 6
  - 3.3.2 Profil Per Variabel Sensor . . . 7
  - 3.3.3 Matriks Korelasi . . . 7
  - 3.3.4 Analisis Interaksi Antar Variabel . . . 8
  - 3.3.5 Analisis Nilai Hilang . . . 8
  - 3.3.6 Sampel Data . . . 9

4 OTA Firmware Server **10**
- 4.1 Gambaran Fitur . . . 10
- 4.2 Halaman OTA Firmware (Build & Upload) . . . 10
- 4.3 Halaman Manajemen Perangkat (Devices) . . . 11
- 4.4 Halaman Build History . . . 12
- 4.5 Remote Console . . . 13

5 Firmware Simulasi ESP32 (Rust) **14**
- 5.1 Gambaran Umum . . . 14
- 5.2 Arsitektur Koneksi . . . 14
- 5.3 Arsitektur Siklus Pengujian Tiga Fase . . . 15
  - 5.3.1 Model Matematika Respons Sensor . . . 15
  - 5.3.2 Parameter Sensor . . . 15
- 5.4 Cuplikan Kode Firmware Rust . . . 16
  - 5.4.1 Fungsi Simulasi Sensor . . . 16
  - 5.4.2 Pengiriman Telemetri ke ThingsBoard via MQTT . . . 16
  - 5.4.3 Remote RPC Control dari ThingsBoard . . . 17
  - 5.4.4 Visualisasi Telemetri pada ThingsBoard Cloud . . . 17

5.5 Output JSON via UART Serial 18
5.6 Cara Build, Flash, dan Hasil Eksekusi 19

6 Lingkungan Pengembangan **21**
6.1 Spesifikasi Mesin Pengembangan 21
6.2 Struktur Direktori Proyek 21
6.3 Kendala Teknis dan Solusi 22

7 Kesimpulan dan Rencana Selanjutnya **23**
7.1 Kesimpulan Progress 23
7.2 Rencana Tahap Selanjutnya 23

Daftar Gambar

2.1 Diagram blok arsitektur tiga komponen sistem E-Nose. 3
2.2 Output terminal saat `run-dev.sh` dijalankan — Rust OTA Server dan Flask Profiler Server aktif bersamaan. 4

3.1 Antarmuka CSV Data Profiler pada `localhost:5000`. 5
3.2 Proses profiling sedang berjalan — tampilan progress 60% pada fase *Generating profiling report*. 6
3.3 Ikhtisar statistik dataset — 9 variabel, 300 observasi, 0 nilai hilang, 0 baris duplikat. 6
3.4 Profil statistik sensor TGS2600 dan MQ-135. 7
3.5 Heatmap korelasi otomatis antar variabel sensor. 7
3.6 Grafik interaksi `tgs2620` terhadap `timestamp`. 8
3.7 Visualisasi *missing values* — semua 9 variabel memiliki 300 nilai lengkap (0% hilang). 8
3.8 Sepuluh baris pertama dataset sensor E-Nose. 9

4.1 Halaman OTA Firmware — upload proyek dan build log real-time. . 10
4.2 Halaman Devices — ESP32-S3 terdaftar dengan status online. 11
4.3 Respons REST API `GET /api/ota/devices`. 12
4.4 Build History — riwayat build firmware proyek ESP32. 12
4.5 Remote Console — monitoring log perangkat live via Wi-Fi. 13

5.1 Dashboard E-Nose pada ThingsBoard Cloud — grafik respons multi-sensor (LPG, NH3, Ethanol, CO, CH4, VOCs), matriks respons radar, indikator suhu/kelembaban DHT22, status koneksi perangkat (Online), serta tombol kontrol RPC (`START` dan `STOP`). 18
5.2 Log terminal Serial Monitor ESP32-S3 — verifikasi koneksi Wi-Fi berhasil mendapatkan IP DHCP, terhubung ke ThingsBoard MQTT, dan pengiriman stream telemetri JSON secara real-time. 20

Daftar Tabel

3.1 Statistik ringkasan dataset sensor E-Nose. 7

4.1 Informasi perangkat ESP32 terdaftar. 11

5.1 Konfigurasi koneksi firmware simulasi ESP32. 14
5.2 Tiga fase siklus pengujian E-Nose dalam firmware Rust. 15
5.3 Parameter baseline dan peak voltage delapan sensor gas (mV). 15

6.1 Spesifikasi mesin pengembangan. 21
6.2 Kendala teknis dan solusi yang diterapkan. 22

7.1 Rencana kerja tahap berikutnya. 23

1. Pendahuluan

1.1 Latar Belakang

Sistem penciuman elektronik (*Electronic Nose* / E-Nose) merupakan instrumen yang meniru kemampuan indera penciuman manusia menggunakan susunan sensor gas dan algoritma pengenalan pola. Teknologi ini memiliki berbagai aplikasi, di antaranya dalam bidang keamanan pangan, deteksi bahan berbahaya, diagnosa medis berbasis bau, serta pemantauan kualitas udara.

Tugas Akhir ini merancang sebuah sistem E-Nose berbasis mikrokontroler **ESP32-S3** yang mengintegrasikan delapan sensor gas (keluarga MQ dan TGS) bersama sensor lingkungan DHT22. Agar pengembangan dapat berlangsung efisien dan terukur, sistem dilengkapi dengan:

- **CSV Data Profiler** — *dashboard* berbasis web untuk analisis eksploratori data (EDA) sensor secara otomatis;
- **OTA (*Over-The-Air*) Firmware Server** — infrastruktur pembaruan *firmware* ESP32 secara nirkabel;
- **Simulasi Telemetri** — firmware Rust (ESP32-S3) yang menghasilkan data sensor sintetik realistis untuk memvalidasi seluruh alur pengiriman data dan komunikasi jaringan selama fase pengembangan perangkat lunak.

1.2 Tujuan

1. Membangun infrastruktur perangkat lunak pendukung untuk sistem E-Nose berbasis ESP32-S3.
2. Menyediakan lingkungan pengembangan terintegrasi yang menggabungkan profiling data CSV, manajemen *firmware* OTA, dan simulasi sensor.
3. Memvalidasi alur pengiriman data telemetri sensor ke platform IoT ThingsBoard Cloud.
4. Menyiapkan fondasi yang kuat untuk arsitektur pemrosesan data dan pembaruan firmware.

1.3 Ruang Lingkup

Laporan progress ini mencakup:

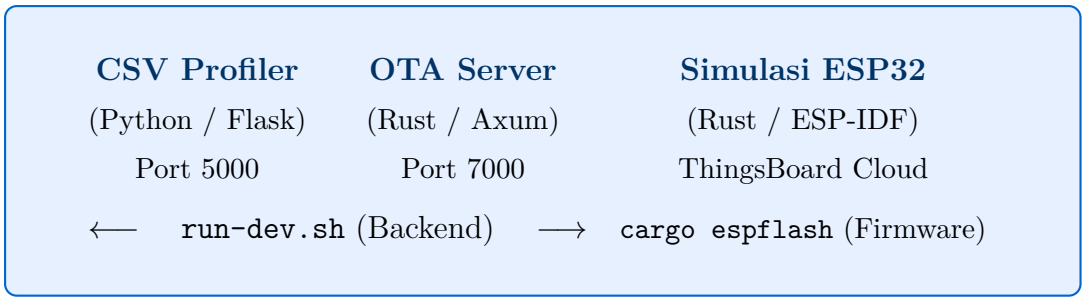
- Pengembangan dan pengujian *CSV Profiling Dashboard*;

- Pengembangan dan pengujian *OTA Firmware Server*;
- Implementasi firmware simulasi telemetri ESP32 berbasis Rust;
- Pengiriman data telemetri ke ThingsBoard Cloud.

2. Arsitektur Sistem

2.1 Gambaran Umum

Sistem yang dikembangkan terdiri atas tiga komponen utama yang saling terhubung, sebagaimana diilustrasikan pada Gambar 2.1.



Gambar 2.1: Diagram blok arsitektur tiga komponen sistem E-Nose.

2.2 Komponen Backend

2.2.1 CSV Profiler Server (Python / Flask)

Server Flask yang berjalan pada **port 5000** bertanggung jawab atas:

- Penerimaan *upload* file CSV dari antarmuka web;
- Pembuatan laporan EDA otomatis menggunakan pustaka `ydata_profiling`;
- Menyajikan laporan profiling secara interaktif melalui browser;
- Menyimpan riwayat sesi profiling.

2.2.2 OTA Firmware Server (Rust / Axum)

Server yang ditulis dengan bahasa **Rust** menggunakan *framework* Axum, berjalan pada **port 7000**. Fungsi utamanya:

- Menerima *upload* proyek firmware (file `.zip`);
- Membangun (*build*) firmware ESP32 secara *server-side*;
- Menyimpan *firmware image* dengan versi dan hash SHA256;
- Melayani permintaan OTA *update* dari perangkat ESP32 yang terdaftar;
- Menyediakan Remote Console untuk monitoring log perangkat secara langsung.

2.2.3 Firmware Simulasi (Rust / ESP32-S3)

File `esp32_simulation/src/main.rs` adalah firmware Rust yang dikompilasi dan di-*flash* langsung ke perangkat ESP32-S3. Firmware ini mensimulasikan logika sensor secara matematis (tanpa ADC fisik), mengontrol motor driver via GPIO, dan mengirimkan telemetri ke ThingsBoard Cloud melalui WiFi MQTT setiap 1 detik. Siklus pengujian juga dapat dikontrol secara nirkabel via ThingsBoard RPC.

2.3 Cara Menjalankan Sistem

Seluruh komponen backend dijalankan sekaligus menggunakan skrip `run-dev.sh`:

```
cd CSV_PROFILING-main
./run-dev.sh
```

Listing 2.1: Perintah menjalankan seluruh sistem.

Setelah berjalan, dua endpoint tersedia:

- **CSV Profiler Dashboard:** http://localhost:5000
- **OTA REST API:** http://localhost:7000

Gambar 2.2 memperlihatkan tampilan terminal saat sistem berhasil dijalankan, dengan kedua server aktif.

```
moreno@Morenos-MacBook-Pro-412 CSV_PROFILING-main % ./run-dev.sh

CSV Profiler: http://localhost:5000
Rust OTA:     http://localhost:7000
Set OTA_PUBLIC_BASE_URL in .env to this PC's LAN address before deployment.
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.25s
     Running `target/debug/csv-profiler-ota-server`
OTA Server running
Local:
http://localhost:7000

LAN / public firmware URL:
http://192.168.110.32:7000
/Users/moreno/Document/Project Coding/TA Rafi Cepmek/CSV_PROFILING-main/python/p
rofiler_server.py:23: DeprecationWarning:
    `import ydata_profiling` is deprecated and will not receive more updates.
    Please install fg-data-profiling via `pip install fg-data-profiling` and use
 `import data_profiling` instead.

  from ydata_profiling import ProfileReport
=====================================================
  CSV Profiler Server  —  http://localhost:5000
=====================================================
 * Tip: There are .env files present. Install python-dotenv to use them.
 * Serving Flask app 'profiler_server'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
 Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.1.69:5000
Press CTRL+C to quit
```

Gambar 2.2: Output terminal saat `run-dev.sh` dijalankan — Rust OTA Server dan Flask Profiler Server aktif bersamaan.

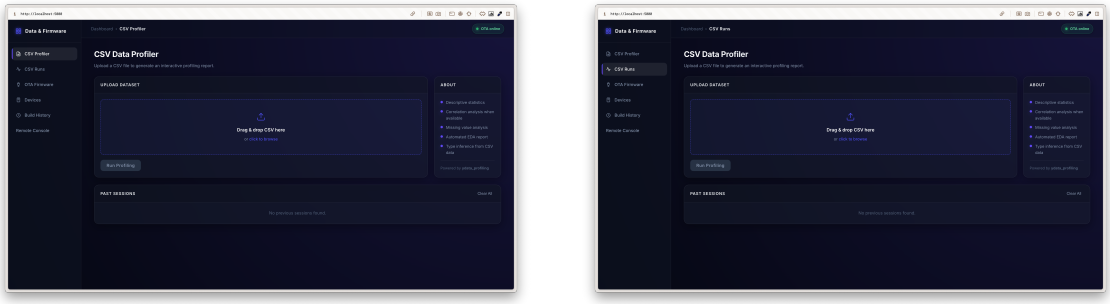
3. CSV Data Profiler

3.1 Gambaran Fitur

CSV Data Profiler adalah antarmuka web yang memungkinkan pengguna mengunggah dataset sensor dalam format CSV dan mendapatkan laporan analisis statistik secara otomatis. Fitur yang tersedia meliputi:

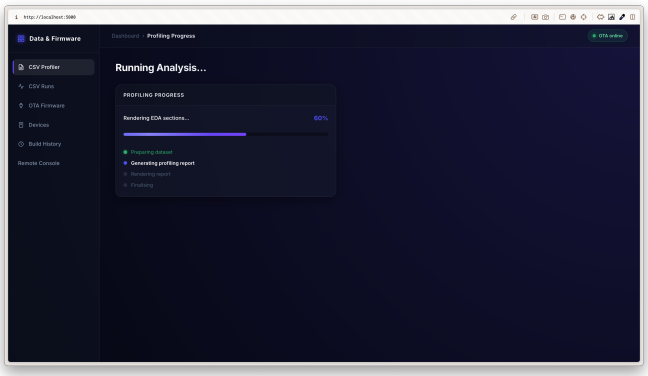
- Statistik deskriptif per variabel (mean, min, max, standar deviasi);
- Analisis korelasi antar variabel sensor;
- Deteksi nilai hilang (*missing values*);
- Inferensi tipe data otomatis;
- Grafik distribusi dan interaksi antar variabel.

3.2 Tampilan Dashboard



(a) Halaman CSV Profiler (upload dataset). **(b)** Halaman CSV Runs (daftar sesi profiling).

Gambar 3.1: Antarmuka CSV Data Profiler pada `localhost:5000`.



Gambar 3.2: Proses profiling sedang berjalan — tampilan progress 60% pada fase *Generating profiling report*.

3.3 Hasil Profiling Dataset Sensor

Dataset yang digunakan dalam pengujian merupakan data simulasi sensor gas E-Nose selama satu siklus pengujian (300 detik, 300 observasi). Dataset mengandung **9 variabel** (1 kolom waktu + 8 sensor gas) dengan **300 baris** data.

3.3.1 Ikhtisar Dataset

Overview

Overview | Alerts 11 | Reproduction

Dataset statistics

Number of variables	9
Number of observations	300
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	21.2 KiB
Average record size in memory	72.4 B

Variable types

Numeric	9

Gambar 3.3: Ikhtisar statistik dataset — 9 variabel, 300 observasi, 0 nilai hilang, 0 baris duplikat.

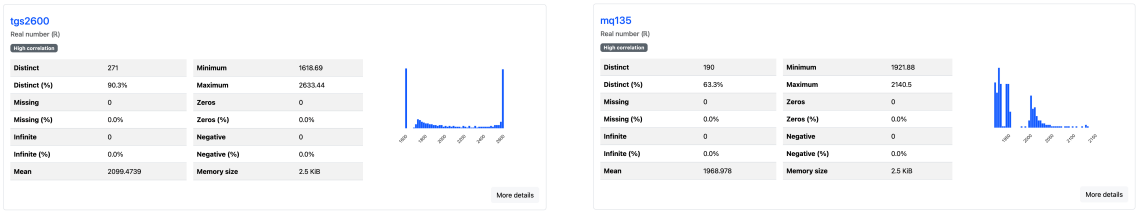
Hasil profiling menunjukkan bahwa dataset dalam kondisi **bersih** sempurna: tidak terdapat nilai hilang (*missing values*) maupun baris duplikat, sehingga data siap digunakan untuk analisis lebih lanjut.

Tabel 3.1: Statistik ringkasan dataset sensor E-Nose.

Properti	Nilai
Jumlah variabel	9
Jumlah observasi	300
Nilai hilang	0 (0.0%)
Baris duplikat	0 (0.0%)
Ukuran total memori	21.2 KiB
Ukuran rata-rata per baris	72.4 B
Tipe variabel	Numerik semua

3.3.2 Profil Per Variabel Sensor

Berikut adalah contoh profil statistik untuk dua sensor representatif.

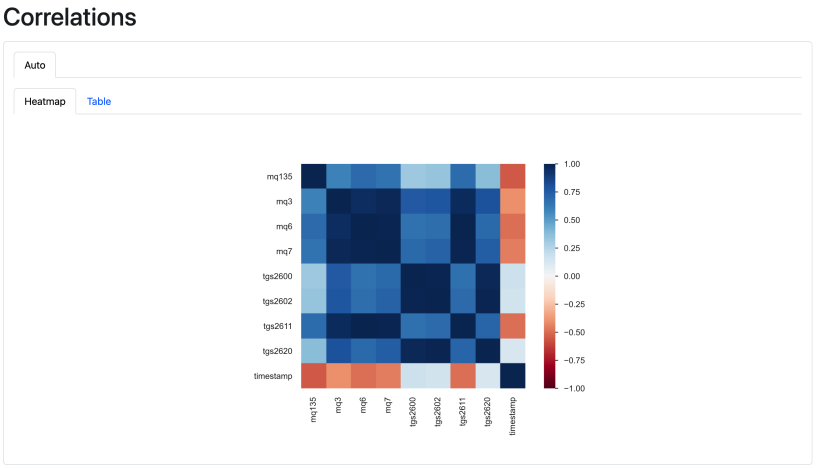


(a) Profil variabel `tgs2600`. **(b)** Profil variabel `mq135`.

Gambar 3.4: Profil statistik sensor TGS2600 dan MQ-135.

Sensor TGS2600 memiliki rentang nilai **1618.69 – 2633.44 mV** dengan rata-rata 2099.47 mV, sementara sensor MQ-135 berkisar pada **1921.88 – 2140.5 mV** dengan rata-rata 1968.98 mV.

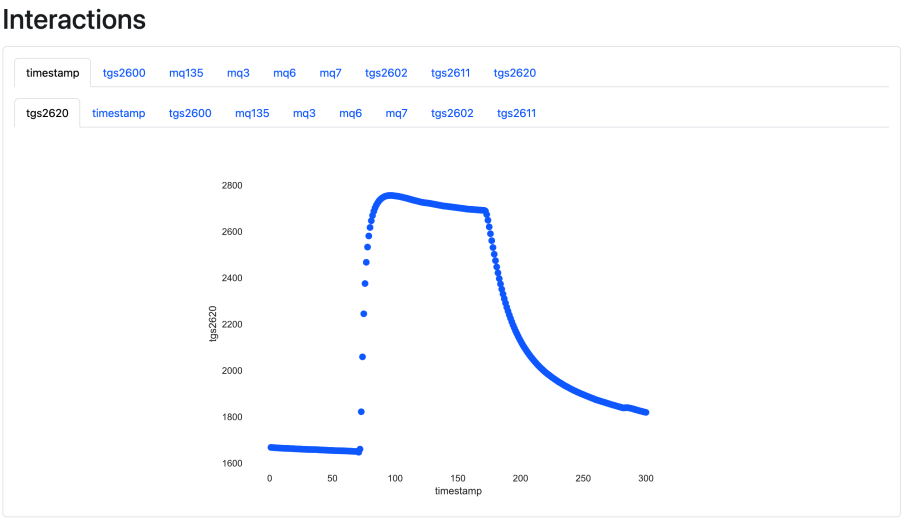
3.3.3 Matriks Korelasi



Gambar 3.5: Heatmap korelasi otomatis antar variabel sensor.

Heatmap korelasi menunjukkan bahwa sebagian besar sensor gas (MQ dan TGS) **berkorelasi positif satu sama lain**, yang konsisten dengan sifat fisik sensor gas yang merespons konsentrasi gas secara bersamaan dalam satu chamber.

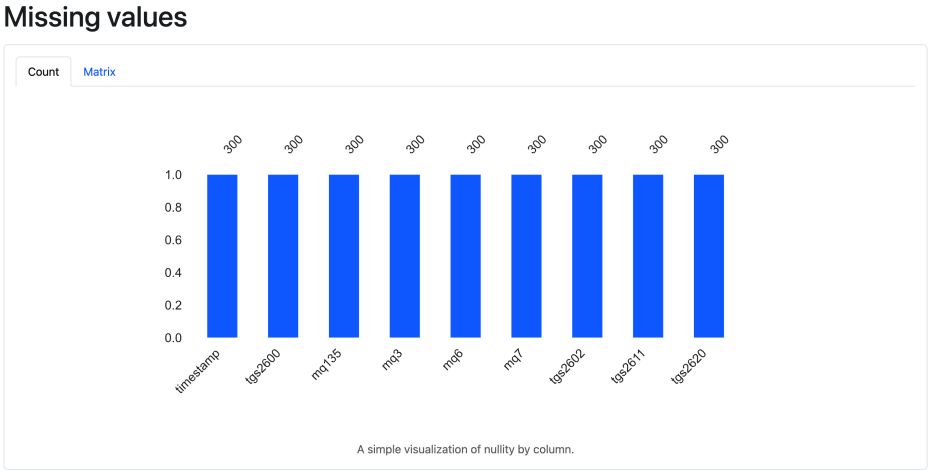
3.3.4 Analisis Interaksi Antar Variabel



Gambar 3.6: Grafik interaksi `tgs2620` terhadap `timestamp`.

Grafik interaksi menampilkan **pola kurva eksponensial** yang khas: nilai sensor naik tajam saat fase inlet dan turun bertahap saat fase outlet.

3.3.5 Analisis Nilai Hilang



Gambar 3.7: Visualisasi *missing values* — semua 9 variabel memiliki 300 nilai lengkap (0% hilang).

3.3.6 Sampel Data

Sample

First rows | Last rows

	timestamp	tgs2600	mq135	mq3	mq6	mq7	tgs2602	tgs2611	tgs2620
0	1	1638.94	1947.75	2630.63	1097.25	3045.19	1422.00	1637.63	1668.00
1	2	1638.38	1948.31	2630.81	1097.25	3045.00	1421.44	1638.00	1667.63
2	3	1637.81	1948.31	2631.00	1097.63	3045.56	1421.25	1637.81	1667.25
3	4	1637.81	1948.31	2631.00	1097.63	3045.56	1420.69	1637.81	1666.69
4	5	1637.25	1948.31	2631.00	1097.81	3045.56	1420.13	1638.00	1666.31
5	6	1636.88	1948.88	2631.19	1097.44	3045.75	1419.94	1638.19	1665.94
6	7	1636.69	1948.88	2631.38	1098.00	3045.75	1419.38	1638.00	1665.56
7	8	1636.13	1948.88	2631.38	1097.81	3045.94	1419.19	1638.00	1665.00
8	9	1635.94	1949.06	2631.56	1097.81	3045.94	1418.63	1638.19	1664.81
9	10	1635.75	1949.63	2631.56	1097.81	3045.94	1418.25	1638.56	1664.63

Gambar 3.8: Sepuluh baris pertama dataset sensor E-Nose.

4. OTA Firmware Server

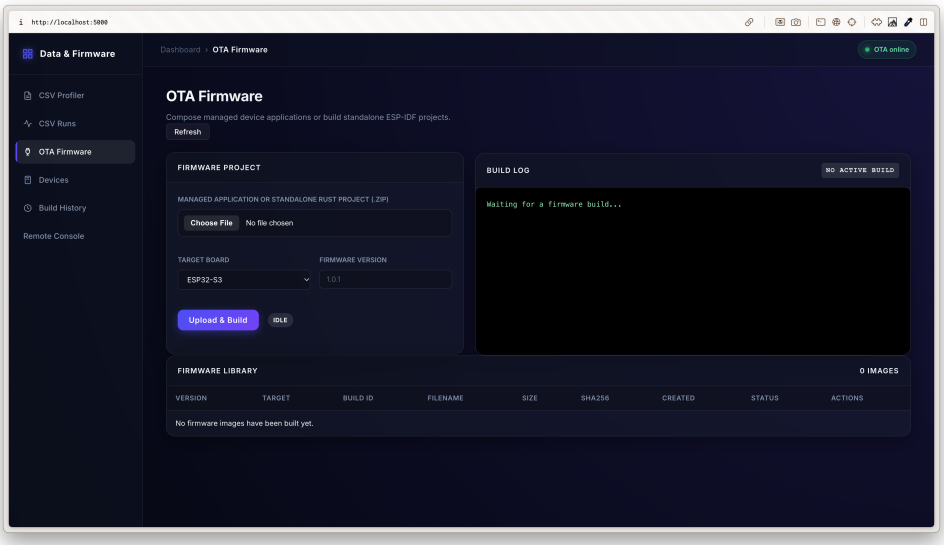
4.1 Gambaran Fitur

OTA (*Over-The-Air*) Firmware Server dibangun menggunakan bahasa **Rust** dengan *framework* Axum. Server ini menyediakan antarmuka terpadu untuk manajemen *firmware* ESP32.

Akses OTA Server

OTA Server berjalan pada `http://localhost:7000` (API backend) dan dapat diakses melalui dashboard web di `http://localhost:5000`.

4.2 Halaman OTA Firmware (Build & Upload)



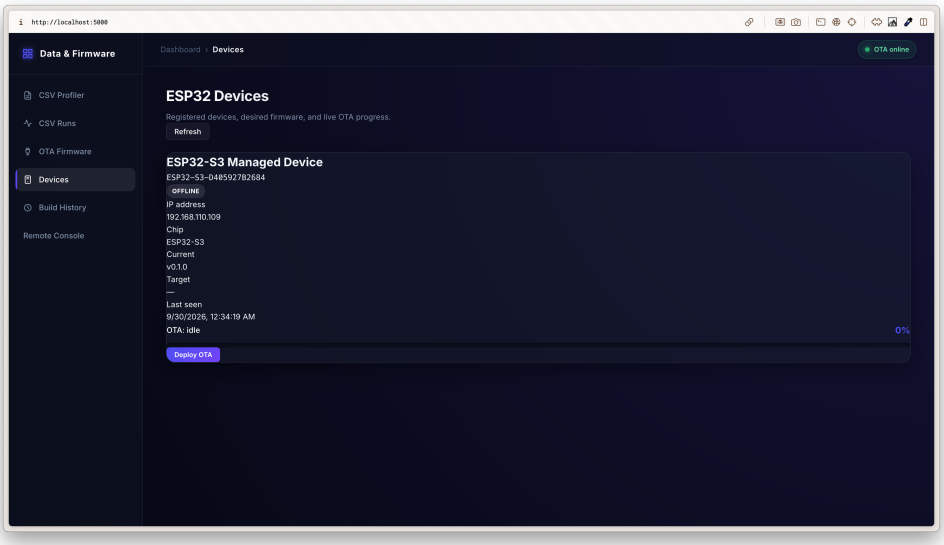
Gambar 4.1: Halaman OTA Firmware — upload proyek dan build log real-time.

Halaman ini menyediakan:

- **Upload file** proyek firmware (`.zip`);
- Pemilihan **Target Board** (ESP32-S3, ESP32, dll.);
- Penentuan **Firmware Version**;
- Tombol **Upload & Build**;
- **Build Log** *real-time*;

- **Firmware Library** — tabel semua *image* yang telah di-build.

4.3 Halaman Manajemen Perangkat (Devices)



Gambar 4.2: Halaman Devices — ESP32-S3 terdaftar dengan status online.

Tabel 4.1: Informasi perangkat ESP32 terdaftar.

Properti	Nilai
Device ID	`ESP32-S3-D405927B2684`
Chip	ESP32-S3
IP Address	192.168.110.109
Versi Firmware	v0.1.0
Status OTA	Idle

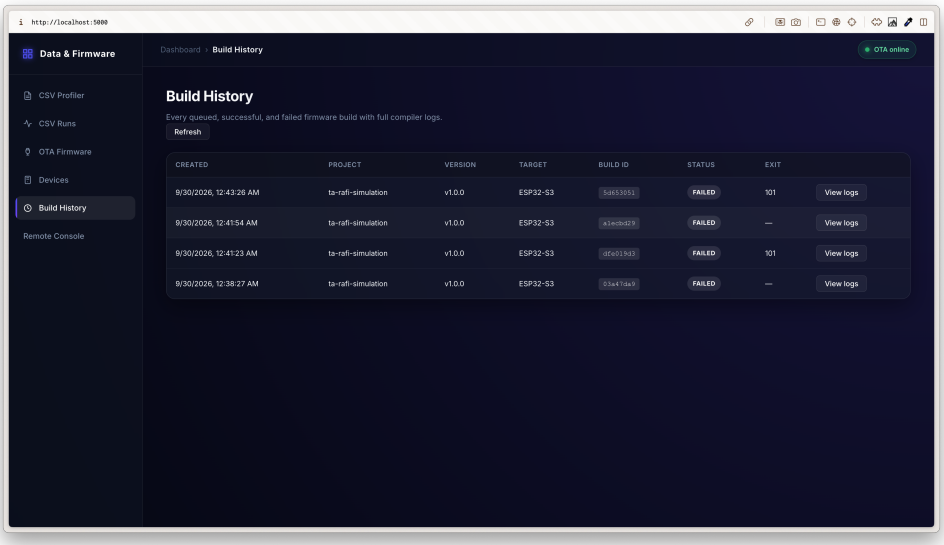
i http://localhost:7000/api/ota/devices

Pretty-print ☑


```
[
  {
    "device_id": "ESP32-S3-D405927B2684",
    "name": "ESP32-S3 Managed Device",
    "chip": "esp32s3",
    "ip_address": "192.168.110.109",
    "current_version": "0.1.0",
    "desired_version": null,
    "desired_firmware_id": null,
    "status": "online",
    "ota_status": "idle",
    "ota_progress": 0,
    "ota_error": null,
    "last_seen": "2026-09-29T17:34:19.127866+00:00"
  }
]
```


Gambar 4.3: Respons REST API `GET /api/ota/devices`.

4.4 Halaman Build History

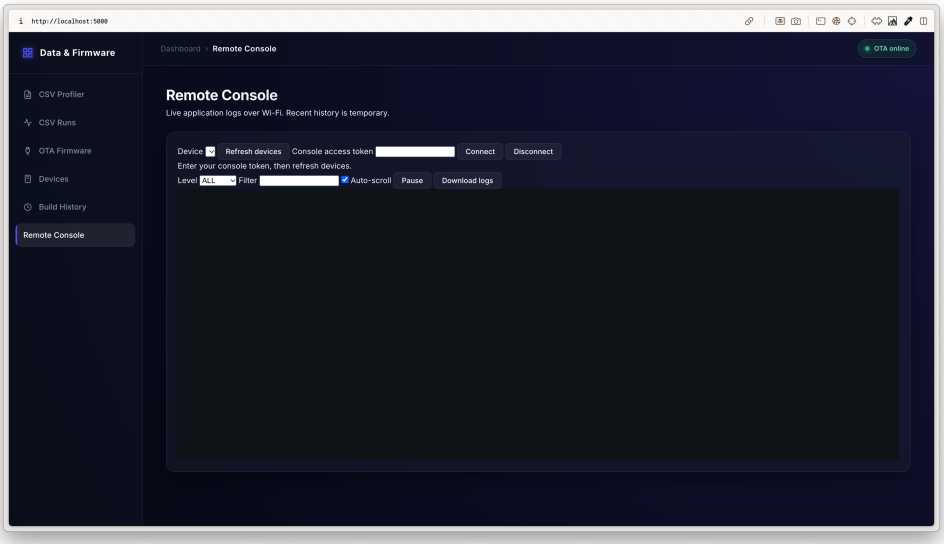


Gambar 4.4: Build History — riwayat build firmware proyek ESP32.

Catatan Build

Build yang tercatat berstatus FAILED disebabkan oleh konfigurasi toolchain ESP-IDF yang belum lengkap. Ini kondisi yang diharapkan pada tahap awal.

4.5 Remote Console



Gambar 4.5: Remote Console — monitoring log perangkat live via Wi-Fi.

Fitur Remote Console memungkinkan pemantauan log aplikasi ESP32 secara *real-time* tanpa koneksi serial/USB, dengan filter level log dan kemampuan download log untuk analisis offline.

5. Firmware Simulasi ESP32 (Rust)

5.1 Gambaran Umum

Firmware simulasi utama yang digunakan dalam sistem E-Nose ditulis menggunakan bahasa **Rust** dan berjalan langsung di atas perangkat keras **ESP32-S3** (`esp32_simulation/src/main.rs`). Firmware ini menggunakan ekosistem `esp-idf-hal` dan `esp-idf-svc` untuk mengakses peripheral hardware secara aman.

Pendekatan ini dipilih karena:

- **Tanpa sensor fisik**: Logika sensor disimulasikan secara matematis langsung di dalam firmware, sehingga tidak perlu ADC atau wiring sensor nyata selama pengembangan awal;
- **Dual connectivity**: Data dikirim secara bersamaan via **USB Serial UART** (untuk monitoring lokal) dan **ThingsBoard MQTT over WiFi** (untuk dashboard cloud);
- **Remote RPC**: Siklus pengujian dapat dikontrol (`START`/`STOP`) dari ThingsBoard Dashboard secara nirkabel melalui MQTT RPC;
- **Motor driver fisik**: Pin GPIO ESP32-S3 dikonfigurasi untuk mengontrol motor driver L298N (inlet/outlet) secara langsung.

5.2 Arsitektur Koneksi

Tabel 5.1: Konfigurasi koneksi firmware simulasi ESP32.

Jalur	Protokol	Fungsi
USB Serial UART	UART0 / stdout	Output JSON telemetri lokal
WiFi + MQTT	TCP port 1883	Kirim telemetri ke ThingsBoard Cloud
MQTT RPC Subscribe	Topic: `v1/devices/me/rpc/request/+`	Terima perintah START/STOP dari dashboard
Driver Motor A (GPIO)	Digital OUT	Inlet pump & valve (pin 5, 17, 14, 19)
Driver Motor B (GPIO)	Digital OUT	Outlet pump & valve (pin 16, 4, 45, 20)

5.3 Arsitektur Siklus Pengujian Tiga Fase

Setiap siklus pengujian berlangsung **300 detik** dalam tiga fase yang dikontrol secara otomatis oleh firmware:

Tabel 5.2: Tiga fase siklus pengujian E-Nose dalam firmware Rust.

Fase	Waktu	Motor Driver	Respons Sensor
1	1 – 70 detik	Driver A & B OFF (Purge)	Baseline
2	71 – 170 detik	Driver A ON (Inlet)	Naik eksponensial
3	171 – 300 detik	Driver B ON (Outlet)	Turun eksponensial

5.3.1 Model Matematika Respons Sensor

Model sensor diimplementasikan sebagai fungsi `simulate_sensors()` dalam Rust. Faktor respons dihitung berdasarkan fase aktif:

- **Fase Inlet (kenaikan eksponensial):**

$$f_{\text{naik}}(t) = 1 - e^{-2.6 \cdot \frac{t-70}{100}}$$

- **Fase Outlet (penurunan eksponensial):**

$$f_{\text{turun}}(t) = e^{-3.2 \cdot \frac{t-170}{130}}$$

- **Nilai tegangan sensor akhir:**

$$V_{\text{sensor}} = V_{\text{baseline}} + (V_{\text{peak}} - V_{\text{baseline}}) \cdot f + \varepsilon, \quad \varepsilon \sim \mathcal{U}(-12, +12)\,\text{mV}$$

Noise sensor ε dibangkitkan menggunakan Linear Congruential Generator (LCG) bawaan tanpa library tambahan, agar firmware tetap minimalis.

5.3.2 Parameter Sensor

Tabel 5.3: Parameter baseline dan peak voltage delapan sensor gas (mV).

Sensor	Baseline (mV)	Peak (mV)
MQ-6	460	1850
MQ-135	530	2400
MQ-3	390	2950
MQ-7	610	1780
TGS2611	430	2380
TGS2602	560	2820
TGS2600	400	2120
TGS2620	490	2640

5.4 Cuplikan Kode Firmware Rust

5.4.1 Fungsi Simulasi Sensor

```rust
fn simulate_sensors(state: &mut SystemState, seed: &mut u32) {
    let elapsed = get_elapsed_time(state);

    let factor = if !state.running || elapsed <= 70 {
        0.0f32  // Fase 1: Standby / Baseline
    } else if elapsed <= 170 {
        // Fase 2: Inlet ON - kurva naik eksponensial
        let progress = (elapsed - 70) as f32 / 100.0;
        1.0 - (-2.6 * progress).exp()
    } else if elapsed <= 300 {
        // Fase 3: Outlet ON - kurva turun eksponensial
        let progress = (elapsed - 170) as f32 / 130.0;
        (-3.2 * progress).exp()
    } else {
        0.0f32
    };

    let noise = |s: &mut u32| rand_range(s, -12.0, 12.0);
    state.sensors.mq6 =
        (460.0 + (1850.0 - 460.0) * factor + noise(seed)).max
    (50.0);
    state.sensors.mq135 =
        (530.0 + (2400.0 - 530.0) * factor + noise(seed)).max
    (50.0);
    // ... (sensor lainnya dengan pola yang sama)
    state.sensors.dht_temp = 27.5 + rand_range(seed, -0.2, 0.2);
    state.sensors.dht_hum  = 62.0 + rand_range(seed, -0.8, 0.8);
}
```

Listing 5.1: Fungsi `simulate_sensors()` — kalkulasi respons sensor per fase dalam Rust (esp32_simulation/src/main.rs).

5.4.2 Pengiriman Telemetri ke ThingsBoard via MQTT

```rust
fn send_thingsboard_telemetry(
    state: &SystemState,
    mqtt: &mut Option<EspMqttClient>
) {
    if let Some(client) = mqtt {
        let payload = format!(
            "{\"mq6\":{:.2}, \"mq135\":{:.2}, ...,
             \"inlet_pump\":{}, \"outlet_pump\":{},
             \"elapsed_time\":{}, \"running\":{}}",
            s.mq6, s.mq135, /* ... */
            state.driver1_active, state.driver2_active,
            get_elapsed_time(state), state.running,
        );
        client.publish(
            "v1/devices/me/telemetry",
            QoS::AtLeastOnce,
```

```
            false,
            payload.as_bytes(),
        ).ok();
    }
}
```

Listing 5.2: Fungsi `send_thingsboard_telemetry()` — publish MQTT ke ThingsBoard Cloud tiap 1 detik.

5.4.3 Remote RPC Control dari ThingsBoard

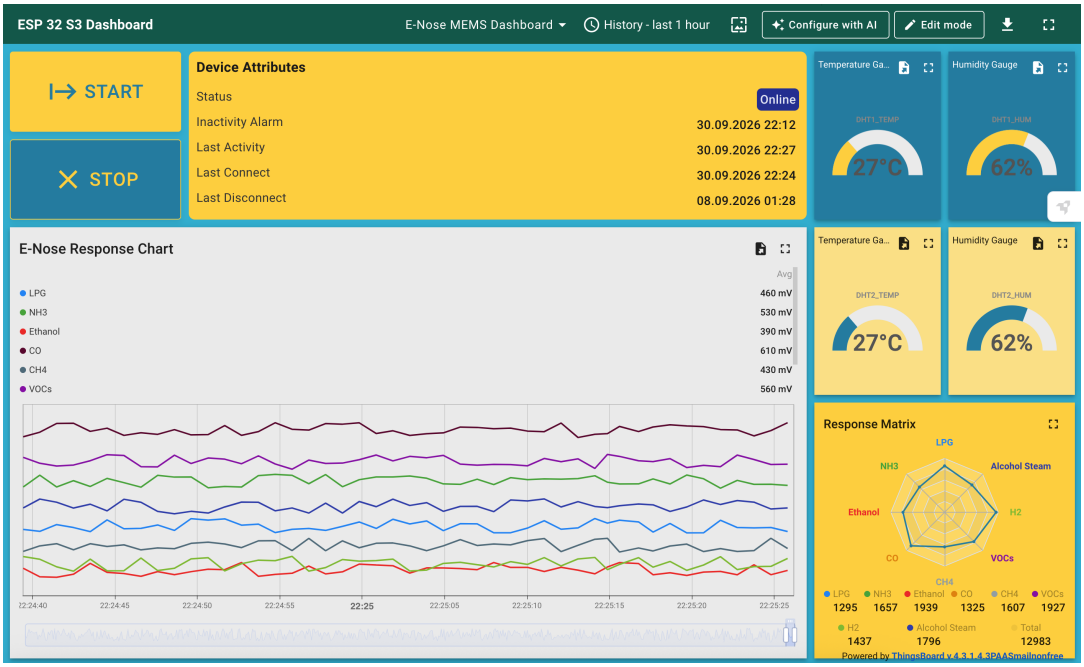
Firmware berlangganan (*subscribe*) ke topic MQTT RPC ThingsBoard untuk menerima perintah `START` / `STOP` dari dashboard:

```
EventPayload::Received { topic, data, .. } => {
    if let Some(t) = topic {
        if t.starts_with("v1/devices/me/rpc/request/") {
            let req_id = &t["v1/devices/me/rpc/request/".len()..];
            let upper = payload_str.to_uppercase();
            if upper.contains("START") {
                rpc_cmd_queue.lock().unwrap().push("START".to_string());
            } else if upper.contains("STOP") {
                rpc_cmd_queue.lock().unwrap().push("STOP".to_string());
            }
            // Kirim balasan RPC
            let resp_topic = format!("v1/devices/me/rpc/response/{}", req_id);
            client.publish(&resp_topic, QoS::AtLeastOnce, false,
                           b"{\"status\":\"ok\"}");
        }
    }
}
```

Listing 5.3: Handler MQTT RPC — menerima perintah START/STOP dari ThingsBoard Dashboard secara nirkabel.

5.4.4 Visualisasi Telemetri pada ThingsBoard Cloud

Data telemetri yang dikirimkan oleh firmware ESP32-S3 diterima dan divisualisasikan secara *real-time* pada antarmuka ThingsBoard Cloud sebagaimana ditunjukkan pada Gambar 5.1.



Gambar 5.1: Dashboard E-Nose pada ThingsBoard Cloud — grafik respons multi-sensor (LPG, NH3, Ethanol, CO, CH4, VOCs), matriks respons radar, indikator suhu/kelembaban DHT22, status koneksi perangkat (Online), serta tombol kontrol RPC (`START` dan `STOP`).

Dashboard ini menyediakan:

- **E-Nose Response Chart**: Grafik kurva dinamis yang memperlihatkan nilai tegangan (mV) masing-masing sensor gas secara kontinu;
- **Response Matrix (Radar Chart)**: Pemetaan sebaran sensitivitas antar gas secara simultan untuk identifikasi profil aroma;
- **Environmental Gauges**: Pemantauan kondisi lingkungan (suhu dan kelembaban relatif) dari sensor DHT22;
- **Device Attributes & RPC Control**: Indikator status perangkat (*Online*) dan tombol interaktif untuk memulai (`START`) atau menghentikan (`STOP`) siklus pengujian dari jarak jauh.

5.5 Output JSON via UART Serial

Selain ThingsBoard, setiap pembacaan sensor juga dikirim melalui USB Serial dalam format JSON terstruktur. Hal ini memudahkan monitoring langsung dari terminal komputer saat debugging:

```
{
  "sensors": {
    "mq6": 1234.56, "mq135": 1876.32, "mq3": 2501.10,
    "mq7": 1102.45, "tgs2611": 1650.00, "tgs2602": 1890.70,
    "tgs2600": 1550.22, "tgs2620": 1750.80,
```

```
    "dht_temp": 27.48, "dht_hum": 62.13
  },
  "status": {
    "inlet_pump": true, "inlet_valve": true,
    "outlet_pump": false, "outlet_valve": false,
    "inlet_time": 45, "outlet_time": 0, "elapsed_time": 115
  },
  "timestamp": 115432
}
```

Listing 5.4: Contoh output JSON dari UART Serial ESP32 tiap 1 detik.

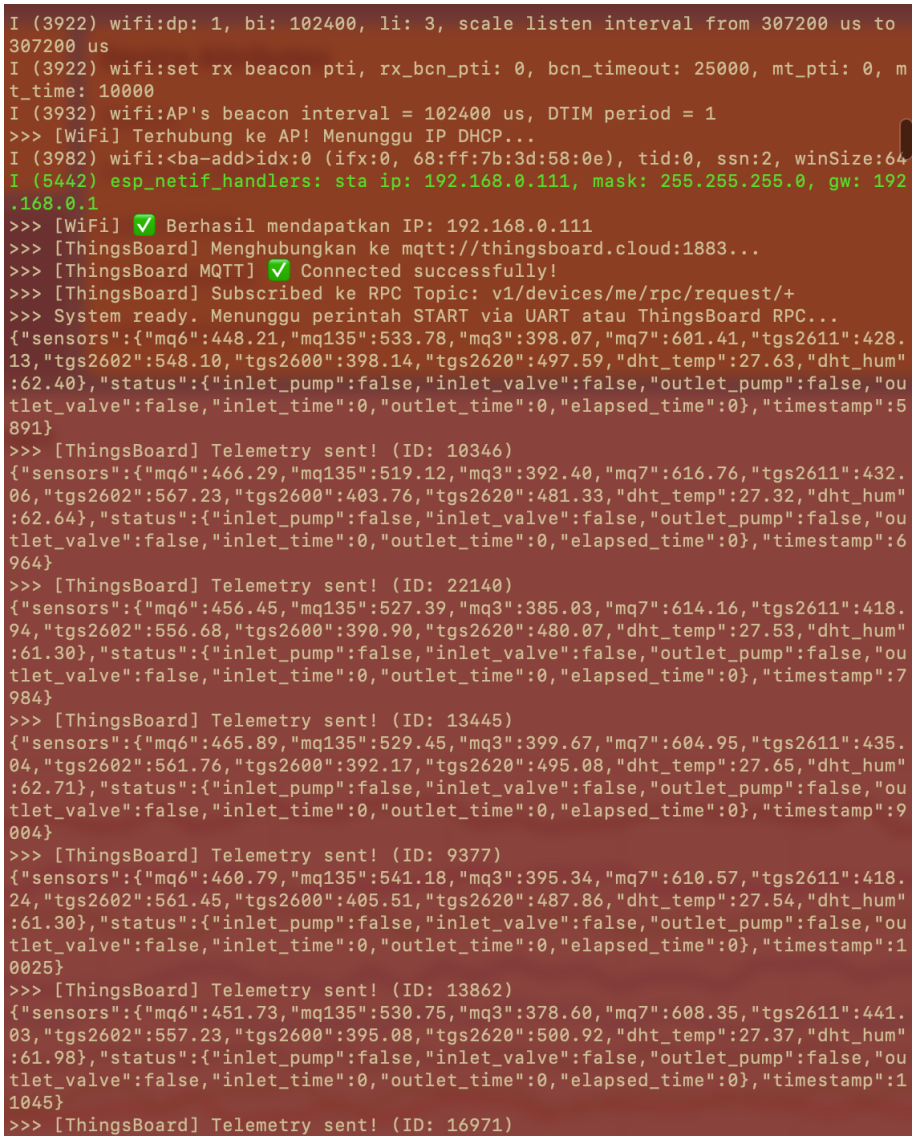
5.6 Cara Build, Flash, dan Hasil Eksekusi

Firmware di-build menggunakan toolchain Rust untuk ESP32 (esp-rs) dan di-flash menggunakan `cargo espflash` dengan menyertakan tabel partisi kustom:

```
cd esp32_simulation
cargo build --release
cargo espflash flash --release --partition-table partitions.csv
   --monitor
```

Listing 5.5: Perintah build dan flash firmware simulasi ke ESP32-S3.

Setelah proses flashing selesai, Serial Monitor pada terminal komputer menampilkan log proses inisialisasi jaringan Wi-Fi, koneksi ke MQTT broker ThingsBoard Cloud, serta aliran telemetri JSON yang berhasil dikirimkan setiap detik (Gambar 5.2).



Gambar 5.2: Log terminal Serial Monitor ESP32-S3 — verifikasi koneksi Wi-Fi berhasil mendapatkan IP DHCP, terhubung ke ThingsBoard MQTT, dan pengiriman stream telemetri JSON secara real-time.

6. Lingkungan Pengembangan

6.1 Spesifikasi Mesin Pengembangan

Tabel 6.1: Spesifikasi mesin pengembangan.

Komponen	Spesifikasi
Sistem Operasi	macOS
Python	3.x (virtual environment `uv`)
Rust / Cargo	Stable channel
Paket Python	Flask, ydata-profiling, pandas, numpy

6.2 Struktur Direktori Proyek

```
TA-Rafi-Cepmek/
|-- CSV_PROFILING-main/      # Dashboard CSV Profiler + OTA
    Server
|    |-- python/             # Flask backend (profiler_server.py)
|    |-- src/                # Rust backend (Axum OTA Server)
|    |-- static/             # Frontend web (HTML/CSS/JS)
|    |-- run-dev.sh          # Script untuk menjalankan semua
    server
|    +-- .env                # Konfigurasi environment
|-- esp32_simulation/        # Firmware simulasi sensor E-Nose
|    |-- src/
|    |    +-- main.rs        # Firmware Rust (ESP32-S3, WiFi,
    MQTT, GPIO)
|    |-- Cargo.toml          # Rust project manifest
|    +-- sdkconfig.defaults  # Konfigurasi ESP-IDF
+-- docs/
     +-- image/              # Screenshot dan gambar dokumentasi
```

Listing 6.1: Struktur direktori proyek E-Nose.

6.3 Kendala Teknis dan Solusi

Tabel 6.2: Kendala teknis dan solusi yang diterapkan.

Kendala	Solusi
`ModuleNotFoundError: flask`	Membuat ulang `venv` dengan `uv` pada macOS (venv lama dibuat di Windows).
`Address already in use (port 7000)`	`lsof -ti :7000 | xargs kill -9`
Deprecation `ydata_profiling`	Dicatat sebagai technical debt; fungsionalitas normal. Migrasi ke `fg-data-profiling` direncanakan.

7. Kesimpulan dan Rencana Selanjutnya

7.1 Kesimpulan Progress

Pada tahap ini, telah berhasil dikembangkan dan diuji tiga komponen infrastruktur:

1. **CSV Data Profiler** — menghasilkan laporan EDA otomatis lengkap termasuk statistik deskriptif, analisis korelasi, dan visualisasi sensor.
2. **OTA Firmware Server** — menyediakan semua fitur manajemen firmware: upload/build, manajemen perangkat, riwayat build, dan remote console.
3. **Simulasi Telemetri** — berhasil mensimulasikan siklus 3 fase dan mengirimkan data ke ThingsBoard Cloud secara real-time.

7.2 Rencana Tahap Selanjutnya

Tabel 7.1: Rencana kerja tahap berikutnya.

No.	Rencana Kerja	Status
1	Pengujian dan validasi flashing firmware ke ESP32	Belum dimulai
2	Integrasi modul klien OTA pada firmware ESP32	Belum dimulai
3	Pengujian OTA update nirkabel via OTA Server	Belum dimulai
4	Implementasi algoritma klasifikasi aroma gas (ML)	Belum dimulai
5	Migrasi ke `fg-data-profiling`	Direncanakan

Laporan ini dibuat pada 30 September 2026 sebagai dokumentasi progress Tugas Akhir. Seluruh kode dan tangkapan layar merupakan hasil kerja nyata pada lingkungan pengembangan penulis.